Chess Code Structure Plan

Files You Need

chess_ai/

```
├── engine.py           # Shared: minimax + alpha-beta search
├── agents.py           # Two evaluation functions + agent wrappers (one per partner)
├── gui.py              # Tkinter GUI (human vs agent, agent vs agent)
├── match_harness.py    # Runs N games between the two agents, logs results
├── main.py             # Entry point — launches GUI
├── requirements.txt    # Dependencies (just python-chess)
├── README.md           # Setup, run instructions, partner attribution
├── match_log.txt       # Generated by match_harness.py (your match report data)
└── docs/
    ├── design.md       # Explanation of minimax + alpha-beta + strategy comparison
    └── match_report.md # Analysis of the agent vs agent results
```

That's 6 code/config files + 2 documentation files.

What Each File Contains

engine.py — Shared, strategy-agnostic

- SearchEngine class
  - search(board, eval_fn) — root-level move selection
  - _minimax(board, depth, alpha, beta, eval_fn) — recursive core with alpha-beta cutoff
  - _evaluate(board, eval_fn) — sign-flip glue for negamax
  - _ordered_moves(board) — capture-first ordering for better pruning
- **Owner:** shared by both partners
- **Depends on:** nothing (only chess)

agents.py — Where the two strategies live

- PIECE_VALUES dict (shared)
- evaluate_materialist(board) + MaterialistAgent — Partner 1
- evaluate_attacker(board) + AttackerAgent — Partner 2
- Both agent classes wrap a SearchEngine and expose choose_move(board)
- **Owner:** Partner 1 owns the first, Partner 2 owns the second
- **Depends on:** engine.py

gui.py — Tkinter GUI

- ChessGUI class
  - Widget builders (mode selector, depth spinbox, board canvas, status bar)
  - _draw_board() / _update_status() — rendering
  - on_click() — human move input
  - _agent_move() / _agent_move_avs() — spawn background threads for agent search
  - _apply_agent_move() — push move + redraw
  - new_game() — reset
- **Owner:** shared (whoever builds it first; the other reviews)
- **Depends on:** engine.py, agents.py, chess

match_harness.py — Tournament runner

- play_game(white_agent, black_agent, max_moves) — one game
- run_match(agent_a, agent_b, games, log_path) — N games with colour alternation + logging
- if __name__ == "__main__": block — runs a 6-game match when executed directly
- **Owner:** shared
- **Depends on:** agents.py

main.py — Entry point

- Construct both agents
- Construct ChessGUI(agent_a, agent_b)
- Call .run()
- **Owner:** shared
- **Depends on:** agents.py, gui.py

requirements.txt

- Single line: python-chess>=1.999

README.md

- Partner names + which agent each owns
- Install instructions
- How to run GUI (human vs agent, agent vs agent)
- How to run match harness
- One-paragraph strategy comparison
- One-paragraph minimax + alpha-beta explanation

docs/design.md

- Deeper write-up: minimax, negamax, alpha-beta, move ordering
- Strategy comparison table (Agent A features vs Agent B features)
- Limitations (no quiescence, fixed depth)

docs/match_report.md

- Scoreline across all games (from match_log.txt)
- Per-game short notes

Dependency Graph (who imports whom)

text

```
main.py
├── agents.py
│   └── engine.py
└── gui.py
    ├── agents.py
    │   └── engine.py
    └── (chess library)


match_harness.py
└── agents.py
    └── engine.py
```

Engine depends on nothing (just chess). Agents depend only on engine. GUI depends on agents + chess. Harness depends on agents. Main wires everything together. **No circular imports**, no cross-coupling between GUI and search.

Build Order (suggested)

1. engine.py — get minimax + alpha-beta working first (test on a couple of positions).
2. agents.py — write both evaluation functions, test each agent plays reasonable chess.
3. match_harness.py — run a 6-game match and confirm the scoreline isn't lopsided to nonsense (e.g. both at 50% or one clearly stronger — either is fine as long as it's real chess).
4. gui.py — wire in the two agents, verify human-vs-agent and agent-vs-agent both run.
5. main.py + requirements.txt + README.md — glue and documentation.
6. docs/ — write the design and match report after the harness produces match_log.txt.

Division of Work Between Partners

Task	Partner 1	Partner 2
`engine.py`	co-write / review	co-write / review
`evaluate_materialist` + `MaterialistAgent`	**owns**	review
`evaluate_attacker` + `AttackerAgent`	review	**owns**
`gui.py`	one builds, other tests both modes	
`match_harness.py`	one builds, other runs match	
`README.md`	attribution + Agent A section	Agent B section
`docs/design.md`	minimax/alpha-beta section	strategy comparison section
`docs/match_report.md`	shared analysis	shared analysis